

Case Study: Video Processing System Using Matrix Multiplication (Strassen's Algorithm)

NAME: P.Manoj
Register no: 192411081
Course code: CSA0609

Title

Video Processing System Using Matrix Multiplication (Strassen's Algorithm)

Description of the Real-World Problem

Modern video processing applications — video editors, streaming platforms, surveillance systems, computer-vision pipelines, and graphics engines — represent each video frame as a large numerical matrix of pixel values. Operations such as rotation, scaling, colour-space conversion, convolution-based filtering, image stabilisation, and 3D transformations are all implemented as matrix multiplications applied repeatedly to every frame.

As video resolution increases (1080p, 4K, 8K), the matrices representing frames and transformation kernels grow very large. Since a typical video plays 24-60 frames per second, the same multiplication must be repeated thousands of times per minute of footage. A naive multiplication routine that scales as $O(n^3)$ quickly becomes the bottleneck of the whole pipeline, causing lag, dropped frames, or unacceptable rendering times.

Divide and Conquer matrix multiplication, and specifically Strassen's Algorithm, reduces the number of scalar multiplications needed, giving a real, measurable speed-up for the large matrices seen in high-resolution video processing.

Problem Statement

Design an efficient matrix multiplication mechanism for a video processing system that transforms high-resolution frames. The solution should:

- Multiply large frame/transformation matrices with minimum computational cost.
- Scale well as frame resolution (matrix size) increases.
- Reduce the number of scalar multiplications compared to the naive method.
- Support recursive decomposition of frames into smaller blocks.
- Maintain numerical correctness while improving speed.

Algorithm Used

Divide and Conquer Matrix Multiplication

The Divide and Conquer approach splits each $n \times n$ matrix into four $(n/2) \times (n/2)$ quadrant sub-matrices, multiplies the quadrants recursively, and combines the partial results to form the final product.

Steps

- Divide matrix A into quadrants A11, A12, A21, A22 and matrix B into B11, B12, B21, B22.
- Recursively compute the required quadrant products.
- Combine the partial products to form the four quadrants of the result matrix C.
- Repeat recursively until the base case (1×1 matrix) is reached.

A direct (naive) Divide and Conquer implementation still needs 8 recursive multiplications per level, so its time complexity remains $O(n^3)$ — identical to the standard triple-loop method. The real gain comes from reducing this count, which is exactly what Strassen's Algorithm does.

Optimized Version — Strassen's Algorithm

Strassen's Algorithm restructures the eight quadrant multiplications into seven cleverly combined products (M1–M7), trading some extra additions/subtractions for fewer, more expensive multiplications:

- $M1 = (A11 + A22) \times (B11 + B22)$
- $M2 = (A21 + A22) \times B11$
- $M3 = A11 \times (B12 - B22)$
- $M4 = A22 \times (B21 - B11)$
- $M5 = (A11 + A12) \times B22$
- $M6 = (A21 - A11) \times (B11 + B12)$
- $M7 = (A12 - A22) \times (B21 + B22)$

The result quadrants are then formed as:

- $C11 = M1 + M4 - M5 + M7$
- $C12 = M3 + M5$
- $C21 = M2 + M4$
- $C22 = M1 + M3 - M2 + M6$

Because multiplication is far more expensive than addition/subtraction for large matrices, cutting the multiplication count from 8 to 7 per recursive level compounds into a significant saving as matrix size grows — exactly the scenario faced with high-resolution video frames.

Computational Challenges in Handling High-Resolution Frames

- Frame matrices for 4K/8K video are extremely large, so even a small constant-factor improvement per multiplication yields large absolute time savings.
- Matrix dimensions are not always a power of two, so padding with zero rows/columns is required before recursive splitting, adding memory overhead.
- Deep recursion increases function-call overhead and stack usage, which must be controlled with a base-case cutoff (switching to naive multiplication below a threshold size).
- Strassen's algorithm needs extra temporary matrices for additions/subtractions, increasing memory bandwidth usage — important for memory-constrained, real-time video hardware.
- Numerical stability can be slightly lower than the naive method because of the extra additions/subtractions, which matters for precision-sensitive colour/pixel data.
- Parallelisation and cache-efficient blocking are needed to fully exploit Strassen's algorithm on modern multi-core / GPU video-processing hardware.

Justification for Choosing the Algorithm (Why Strassen's Algorithm?)

- Reduces asymptotic time complexity from $O(n^3)$ to approximately $O(n^{2.781})$.
- Fewer scalar multiplications translate directly into faster frame transformations at scale.
- Recursive, divide-and-conquer structure maps naturally onto block-based frame processing.
- Well suited to large matrices — exactly the case for high-resolution video frames.
- Can be combined with a naive-multiplication cutoff for small blocks to avoid recursion overhead on small sub-matrices.

Complexity Analysis

Method	Time Complexity	Space Complexity
Naive (Triple Loop) Multiplication	$O(n^3)$	$O(n^2)$

Basic Divide and Conquer (8 multiplications)	$O(n^3)$	$O(n^2)$ + recursion stack
Strassen's Algorithm (7 multiplications)	$O(n^{2.781})$	$O(n^2)$ (extra temp matrices)

Explanation

Naive Multiplication

- Uses three nested loops; every output element needs n multiplications, giving n^3 total.
- Simple and numerically stable, but does not scale well for large frame matrices.

Basic Divide and Conquer

- Splits the problem into 8 sub-problems of size $n/2$, each combined in $O(n^2)$ time.
- Recurrence $T(n) = 8T(n/2) + O(n^2)$ still resolves to $O(n^3)$.

Strassen's Algorithm

- Splits the problem into only 7 sub-problems of size $n/2$.
- Recurrence $T(n) = 7T(n/2) + O(n^2)$ resolves to $O(n^{\log_2 7}) \approx O(n^{2.781})$.
- For large n (large, high-resolution frames), this is measurably faster than the cubic methods.

Manual Execution (Step-by-Step Process)

Consider two small 2×2 blocks taken from a larger frame-transformation matrix:

$$A = \begin{bmatrix} 1 & 3 \\ 7 & 5 \end{bmatrix} \quad B = \begin{bmatrix} 6 & 8 \\ 4 & 2 \end{bmatrix}$$

Step 1: Compute M1–M7

- $M1 = (A11+A22) \times (B11+B22) = (1+5) \times (6+2) = 6 \times 8 = 48$
- $M2 = (A21+A22) \times B11 = (7+5) \times 6 = 12 \times 6 = 72$
- $M3 = A11 \times (B12-B22) = 1 \times (8-2) = 1 \times 6 = 6$
- $M4 = A22 \times (B21-B11) = 5 \times (4-6) = 5 \times (-2) = -10$
- $M5 = (A11+A12) \times B22 = (1+3) \times 2 = 4 \times 2 = 8$
- $M6 = (A21-A11) \times (B11+B12) = (7-1) \times (6+8) = 6 \times 14 = 84$
- $M7 = (A12-A22) \times (B21+B22) = (3-5) \times (4+2) = -2 \times 6 = -12$

Step 2: Combine Results

- $C11 = M1+M4-M5+M7 = 48-10-8-12 = 18$
- $C12 = M3+M5 = 6+8 = 14$
- $C21 = M2+M4 = 72-10 = 62$
- $C22 = M1+M3-M2+M6 = 48+6-72+84 = 66$

Final Result

$$C = A \times B = \begin{bmatrix} 18 & 14 \\ 62 & 66 \end{bmatrix}$$

This matches the result obtained from direct (naive) multiplication, confirming Strassen's algorithm is correct while using only 7 multiplications instead of 8.

Screenshot of the Program

```
strassen.py - Terminal

Code (Strassen's Recursive Multiplication):
def strassen(A, B):
    n = len(A)
    if n == 1:
        return [[A[0][0]*B[0][0]]]
    A11, A12, A21, A22 = split(A)
    B11, B12, B21, B22 = split(B)

    M1 = strassen(add(A11, A22), add(B11, B22))
    M2 = strassen(add(A21, A22), B11)
    M3 = strassen(A11, sub(B12, B22))
    M4 = strassen(A22, sub(B21, B11))
    M5 = strassen(add(A11, A12), B22)
    M6 = strassen(sub(A21, A11), add(B11, B12))
    M7 = strassen(sub(A12, A22), add(B21, B22))

    C11 = add(sub(add(M1, M4), M5), M7)
    C12 = add(M3, M5)
    C21 = add(M2, M4)
    C22 = add(sub(add(M1, M3), M2), M6)

    return combine(C11, C12, C21, C22)

Output:
Matrix A (Frame Transform 1):
[8, 6, 3, 3]
[7, 1, 7, 5]
[1, 9, 2, 5]
[0, 5, 3, 9]

Matrix B (Frame Transform 2):
[5, 1, 1, 1]
[5, 3, 9, 7]
[5, 4, 4, 2]
[8, 6, 4, 8]

Naive Multiplication Result:
[109, 56, 86, 80]
[115, 68, 64, 68]
[100, 66, 110, 108]
[112, 81, 93, 113]

Strassen's Multiplication Result:
[109, 56, 86, 80]
[115, 68, 64, 68]
[100, 66, 110, 108]
[112, 81, 93, 113]

Results Match: True
```

Output Screenshot

Example output (4×4 frame-transform matrices):

Matrix A (Frame Transform 1): [8,6,3,3] [7,1,7,5] [1,9,2,5] [0,5,3,9]

Matrix B (Frame Transform 2): [5,1,1,1] [5,3,9,7] [5,4,4,2] [8,6,4,8]

Naive Result: [109,56,86,80] [115,68,64,68] [100,66,110,108] [112,81,93,113]

Strassen's Result: [109,56,86,80] [115,68,64,68] [100,66,110,108] [112,81,93,113]

Results Match: True

Applications

- High-resolution video editing and rendering software
- Real-time video filtering and colour-grading pipelines
- Image and frame rotation, scaling, and warping in graphics engines
- Computer vision and convolutional neural network frame processing
- Video stabilisation and motion-compensation systems
- 3D animation and rendering transformations
- Surveillance and streaming systems processing large frame batches

Advantages

- Fewer scalar multiplications than the naive $O(n^3)$ approach.
- Scales better for the large matrices seen in high-resolution frames.
- Recursive block structure fits naturally with frame-tiling / block-processing pipelines.
- Can be combined with parallel processing for further speed-up.
- Provides a genuine asymptotic improvement, not just a constant-factor tweak.

Limitations

- Higher constant factors and overhead make it slower than naive multiplication for small matrices.
- Requires extra memory for temporary sub-matrices (M1–M7).
- Matrix dimensions must be padded to the nearest power of two, adding overhead.
- Slightly reduced numerical stability due to additional additions/subtractions.
- Recursive implementation increases code complexity compared to the straightforward triple loop.

Comparison with Other Methods

Method	Time Complexity	Multiplications (per split)	Extra Memory
Naive Multiplication	$O(n^3)$	8 (implicit)	Low
Basic Divide and Conquer	$O(n^3)$	8	Moderate
Strassen's Algorithm	$O(n^{2.781})$	7	Higher
Coppersmith–Winograd (theoretical)	$O(n^{2.378})$	Fewer (complex)	Very High

Why Strassen's Algorithm is Better for This Case Study

- Directly targets the $O(n^3)$ bottleneck that limits naive multiplication on large frame matrices.
- Provides genuine asymptotic gains that grow larger as resolution (and matrix size) increases.
- Recursive block decomposition aligns naturally with how video frames are already tiled/processed.
- More practical than further theoretical algorithms (e.g. Coppersmith–Winograd), which have very high constant factors and are not used in real systems.

Conclusion

A video processing system that transforms high-resolution frames needs a matrix multiplication strategy that scales well as frame size grows. The naive triple-loop method and the basic Divide and Conquer approach both run in $O(n^3)$ time, which becomes a serious bottleneck for 4K/8K frame matrices processed dozens of times per second.

Strassen's Algorithm, an optimised Divide and Conquer technique, reduces the number of recursive multiplications from 8 to 7 per split, lowering the overall time complexity to approximately $O(n^{2.781})$. While it introduces extra additions, temporary matrices, and padding overhead — making it less efficient for small matrices — its asymptotic advantage becomes increasingly valuable as matrix size grows, which is precisely the situation in high-resolution video processing. Combined with a naive-multiplication cutoff for small sub-matrices, Strassen's algorithm offers a practical, scalable solution for optimising matrix multiplication in real-world video processing systems.